



Maastricht University

Department of Advanced Computing Science

Algorithmic Design

Week 1:

Introduction, Greedy Algorithms, and More!

David Mestel and **Steven Chaplick**

2025-2026 Period 5

Practicalities

- Steven Chaplick (office: PHS1 C4.008, s.chaplick@maastrichtuniversity.nl)
- David Mestel (office: PHS1 C4.008, david.mestel@maastrichtuniversity.nl)

Lectures:

- Tuesdays & Thursdays, check **timetable** sometimes we have a break!
- Materials posted on Canvas under Modules

Tutorials:

- Thursdays (week two), check **timetable**, sometimes we have a break!
- Your opportunity to ask questions, work on problem sets, collaborate, and clarify course material

Assessment

- One Test (in tutorial): 30min closed-book. This will occur **in week 3**.
- Final Exam : 2 hours closed book.
- Final grade = $\max \begin{cases} 100\% \text{ Final Exam} \\ 10\% \text{ Test} + 90\% \text{ Final Exam} \end{cases}$

What Is This Course About?

Background: Data Structures and Algorithms, Discrete Mathematics

Data Structures

- How to organize and perform operations on data in memory

Algorithms:

- Methods use data structures to implement functionality.

Complexity Analysis:

- How to measure the “speed” of an algorithm.

Here: deeper look into algorithmic paradigms, and when tasks are inherently difficult to solve computationally.

We will assume:

- You know how to build/use data structures
- You can do basic complexity analysis $\rightsquigarrow$ asymptotic runtime, Big- $\mathcal{O}$
- Can apply standard proof techniques $\rightsquigarrow$ cases, contradiction, induction, etc.

What Is This Course About?

To prove a statement is correct various techniques can be used:

- Proof by cases, proof by contradiction, induction, etc.

Picking the different methods can make the statement easier or more difficult to prove.

- Learning these general techniques gives us tools to find the “easy” proof.

Computational tasks can be similarly approached via different algorithmic paradigms.

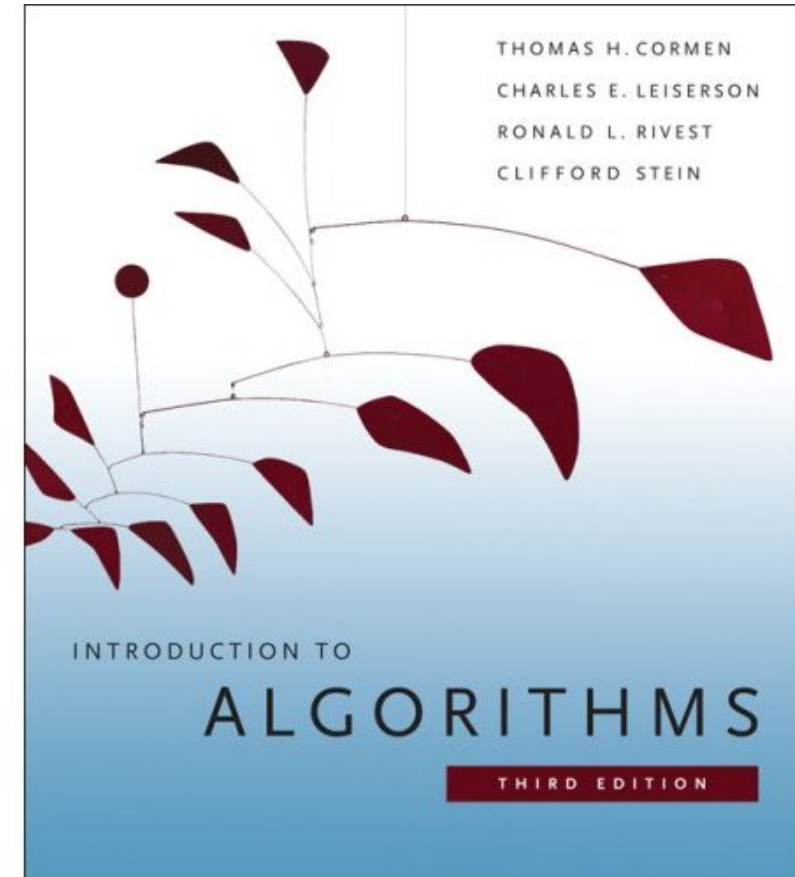
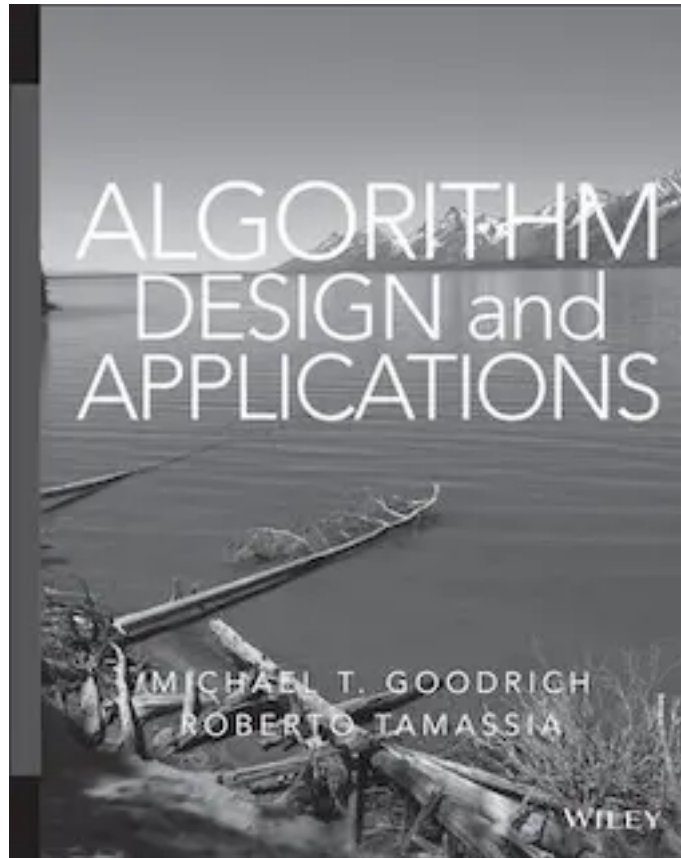
- Greedy, divide-and-conquer, dynamic programming, etc.

Course Outline:

- Start with these paradigms and corresponding tasks that can be solved with them
- Some tasks lack efficient algorithms to solve them, why?
Classification of NP-hard problems.
- But we still need to tackle those problems
 $\rightsquigarrow$ Backtracking, Linear Programming, Branching-and-Bound, etc.

Books

The 'official' textbook for the course is [GT]
Goodrich & Tamassia, 'Algorithm Design and
Applications'

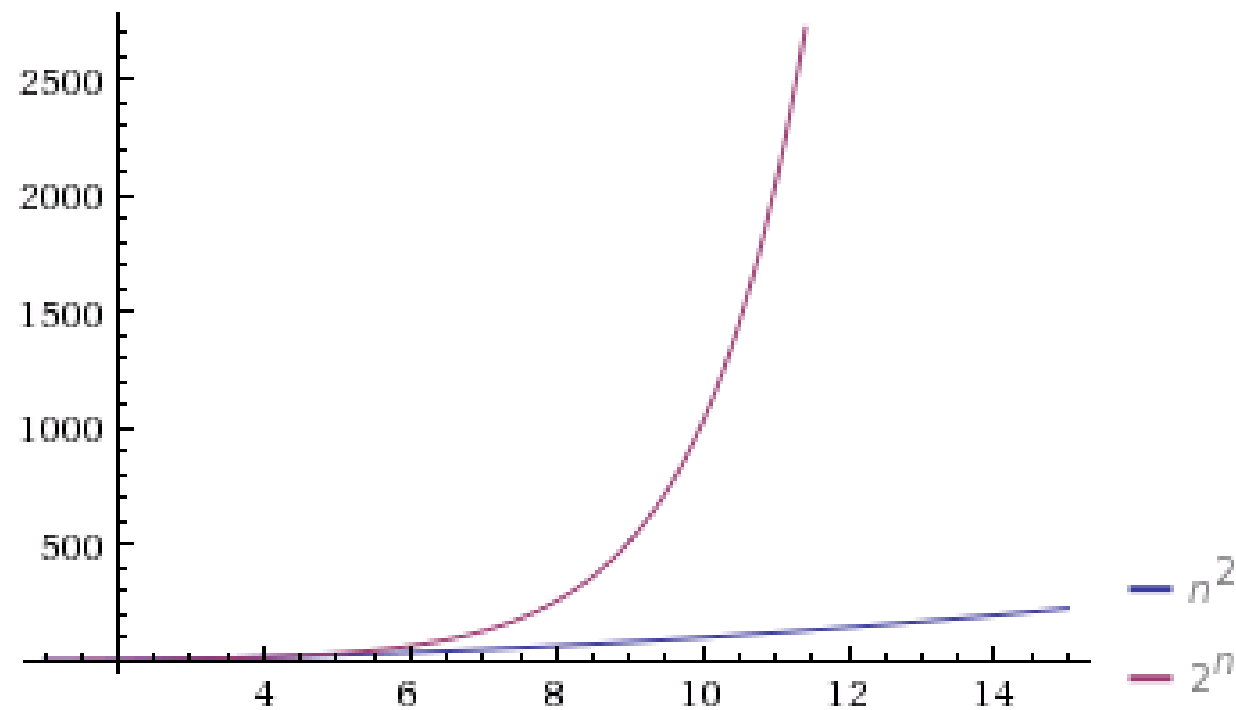


(additional standard book)
'Introduction to Algorithms'
[CLRS]

Context

■ efficient vs. inefficient algorithms

⇒ polynomial vs. super-polynomial (exponential) algorithms



Computed by Wolfram|Alpha

Better Hardware vs. Better Algorithms?

Hardware: time / operation.

Algorithms: operations / job.

Week 1:

Greedy Algorithms: [GT §10, CLRS §16]

- Idea: Pick what looks the best step-by-step.
- Scheduling two ways [GT §10.2], [CLRS §16.1]

Greedy Algorithms on Graphs:

- Minimum Spanning Trees [GT §15] [CLRS §23], By Prim [GT §15.3] [CLRS §23.2]
- Shortest Paths [GT §14] [CLRS §24], By Dijkstra [GT §14.2] [CLRS §24.3]

Reminder: Asymptotic Analysis [GT 1.1.5, 1.1.6], [CLRS 3.1]

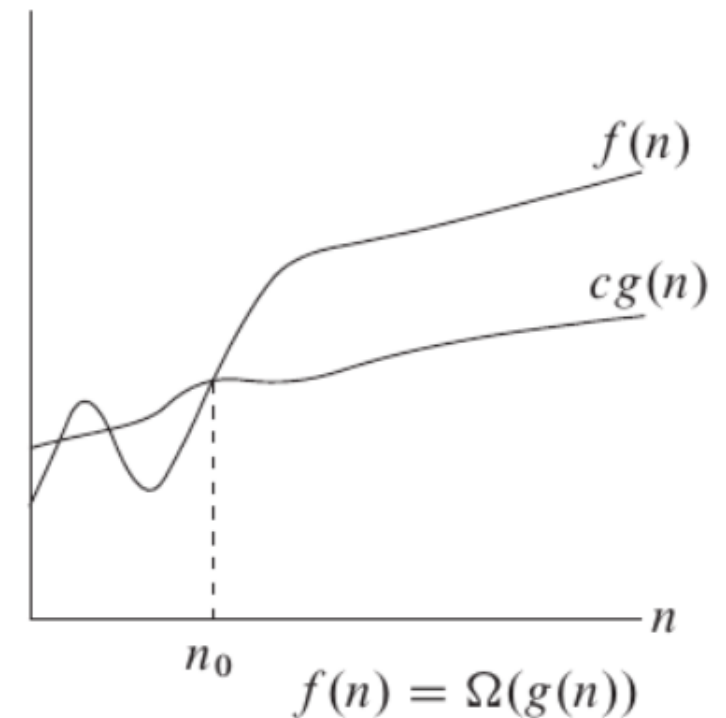
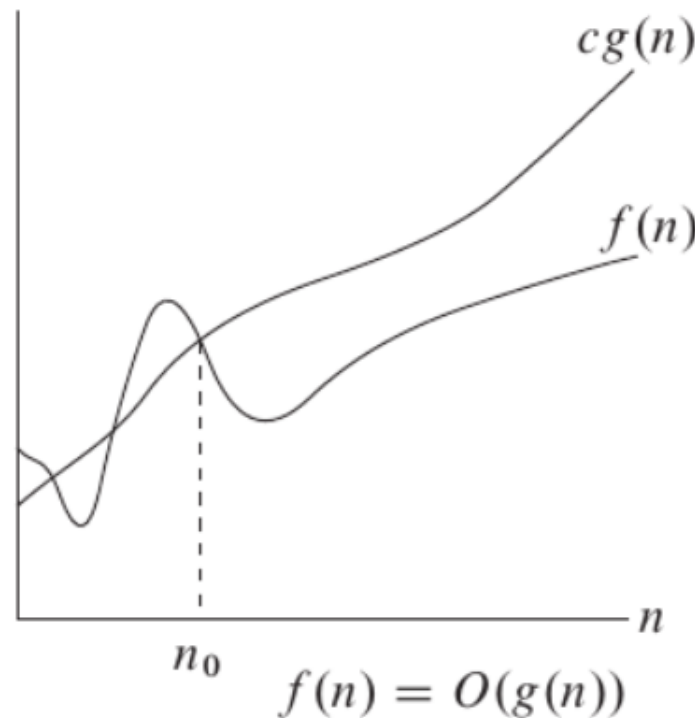
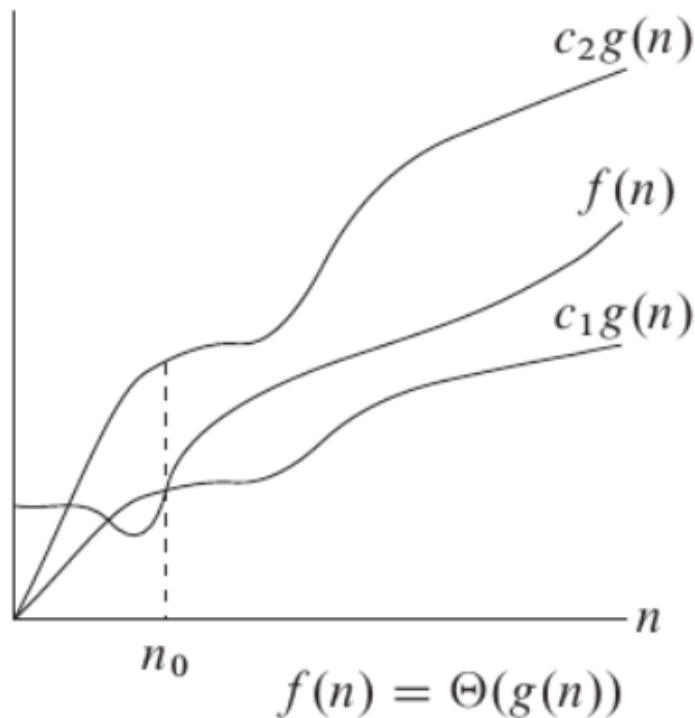
Represent the time taken by an algorithm on an input of **size** n , by a function $f : \mathbb{N} \rightarrow \mathbb{N}$

$f(n) = 10^{10}n$, linear, $\in O(n)$. $\leftarrow$ kind of cheating here

$f(n) = 25n^3 + 5n^2 + \log(n)$, cubic, $\in O(n^3)$

Idea: for “big enough” input the constants are not important.

Worst-Case Analysis: describe the function f according to the **slowest** input of size n .



Greedy algorithms

- **Greedy** means making the choice that seems best now, without thinking about the future
- Algorithmically, “do the what looks the best each step”
- But what does **seems best now** mean? Need to pick the right measure of “best”.

For example, you come across a pile of coins on a table. You can only take 10 coins. How can you maximize the amount of money you get?

A simple greedy algorithm, for a given set of coins:

1. Pick up a coin of maximum value.
2. If you have taken 10 coins, stop. If not, go to 1

did you spot the bug here?

Picking Coins

Input: A non-empty set of coins $C = \{c_1, \dots, c_n\}$, and a function $v : C \rightarrow \mathbb{N}$ where for each coin $c \in C$, $v(c)$ is the value of c . A number t of coins to be taken where $0 \leq t \leq n$.

Goal: Take t of coins of maximum total value.

A simple greedy algorithm:

1. Pick up a coin of maximum value.  1. Pick c_i in C such that v_i is maximum.
2. If t coins picked, stop. Otherwise, go to 1

Why is this **optimal**? An *optimal algorithm*: for any input, output is *optimal* (**best possible**)

Suppose $\text{OPT} \subseteq C$ is an **optimal** selection of t coins.

For any set $S \subseteq C$ where $|S| = t$, $\sum_{c \in S} v(c) \leq \sum_{c \in \text{OPT}} v(c)$

Let $S \subseteq C$ be the t coins picked by greedy.

Suppose (for a contradiction) that the greedy choice is not optimal.

Thus, OPT and S differ in value. Since both have t coins, OPT contains coin c' of value different from some coin in S . $\Rightarrow v(c') \leq v(c)$ for every $c \in S$ [by the Greedy Rule].

Further, let c be a coin of maximum value in S but not in OPT . Thus, $v(c) > v(c')$.

Now: Replacing c' by c in OPT **raises** the value of OPT . Contradiction!

Picking Coins: now with pseudocode and runtime analysis

Input: A non-empty set of coins $C = \{c_1, \dots, c_n\}$, and a function $v : C \rightarrow \mathbb{N}$ where for each coin $c \in C$, $v(c)$ is the value of c . A number t of coins to be taken where $0 \leq t \leq n$.

Goal: Take t of coins of maximum total value.

A simple greedy algorithm:

1. Pick up a coin of maximum value.  1. Pick c_i in C such that v_i is maximum.
2. If no coins remain or t coins picked, stop. Otherwise, go to 1

Pseudocode: Repeatedly pick the max.

Initialize $R = \emptyset$ as the coins we will return.

For i from 1 to t

■ Let c be the coin in C of maximum value. $O(n)$

■ Remove c from C , and add it to R . $O(1)$

End For (C as linked list)

Timing: t iterations of the loop, each taking $O(n)$ steps. $\rightsquigarrow O(tn)$ in total.

Better way?

Sort C by value so that $C = (c_1, \dots, c_n)$

where $v_1 \geq v_2 \geq \dots \geq v_n$.

Return $R = (c_1, \dots, c_t)$.

Timing: Sorting takes $O(n \log n)$ steps, followed by picking t elements $O(1)$.

$O(n \log n)$ in total.

Exercise Make it faster: $O(n \log t)$.

Is greedy always good?

Suppose our coins not only have a **value**, but also a **weight**.

And, instead of limiting the number of coins to pick (e.g., 10), we limit the total **weight** of the coins we pick.

KNAPSACK

Input: Given a weight capacity $W \geq 0$ and n objects with non-negative weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$ respectively, i.e., object i has weight w_i and value v_i .

Goal: Pick a subset S of the given objects of maximum value and total weight at most W .

Greedy approach? ■ Pick coins by max value until weight limit met.

Sanity check? (try some small examples)

How many objects to try?

~~One?~~ ~~Two?~~ Three? With three there is a chance of making a bad choice.

Consider 3 coins where the (w_i, v_i) are $(2,4)$, $(1,3)$, $(1,3)$, and capacity $W = 2$.

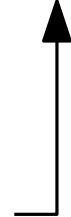
KNAPSACK

Input: Given a weight capacity $W \geq 0$ and n objects with non-negative weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$ respectively, i.e., object i has weight w_i and value v_i .

Goal: Pick a subset S of the given objects of maximum value and total weight at most W .

3 coins where the (w_i, v_i) are $(2,4)$, $(1,3)$, $(1,3)$, and capacity $W = 2$.

Another greedy approach?

■ Pick coins by *value density*: $\frac{v_i}{w_i}$  works for this :) Can we break this too?

Consider 3 coins where the (w_i, v_i) are $(3,5)$, $(2,3)$, $(2,3)$, and capacity $W = 4$.

Greedy gets the first coin (value= 5), but then cannot take any more.

It is better to take the other two coins (value= 6).

We need a more powerful technique (Dynamic Programming → week 3).

Or an easier problem, see FRACTIONAL KNAPSACK [GT §10.1]

When can we use a greedy approach?

Optimization problem with a set of **configurations** and an **objective function**, and the goal is to find a configuration maximizing (or minimizing) the value of the objective function.

Outline of a greedy algorithm:

- View the problem as a series of choices
- At every point, make the choice that most improves the objective function
- Do this until you have made enough choices to have a solution

Each problem can have **many** viable greedy algorithms

Let's try some scheduling problems.

Problem: Task Scheduling [GT §10.2] (aka interval scheduling)

- Streaming a collection of live **events** in the fewest **channels**.
- Given: a set of events by time slots $T_1, \dots, T_n$, where
 - each T_i has a start and finish time, (s_i, f_i) , $s_i < f_i$.
- Objective: Minimize the number of streaming channels needed to show all events.
- Notes:
 - Fixed cost per streaming channel regardless of the amount of time it needs to be active.
 - The output should indicate which event is on which channel.

What is a schedule/plan? Function $p : \text{events} \rightarrow \text{channels}$ (e.g., represented numbers).

When is a schedule **feasible**? No **conflicting** events are mapped to the same room.
That is, if $p(T_i) = p(T_j)$ then either $f_i \leq s_j$ or $f_j \leq s_i$.

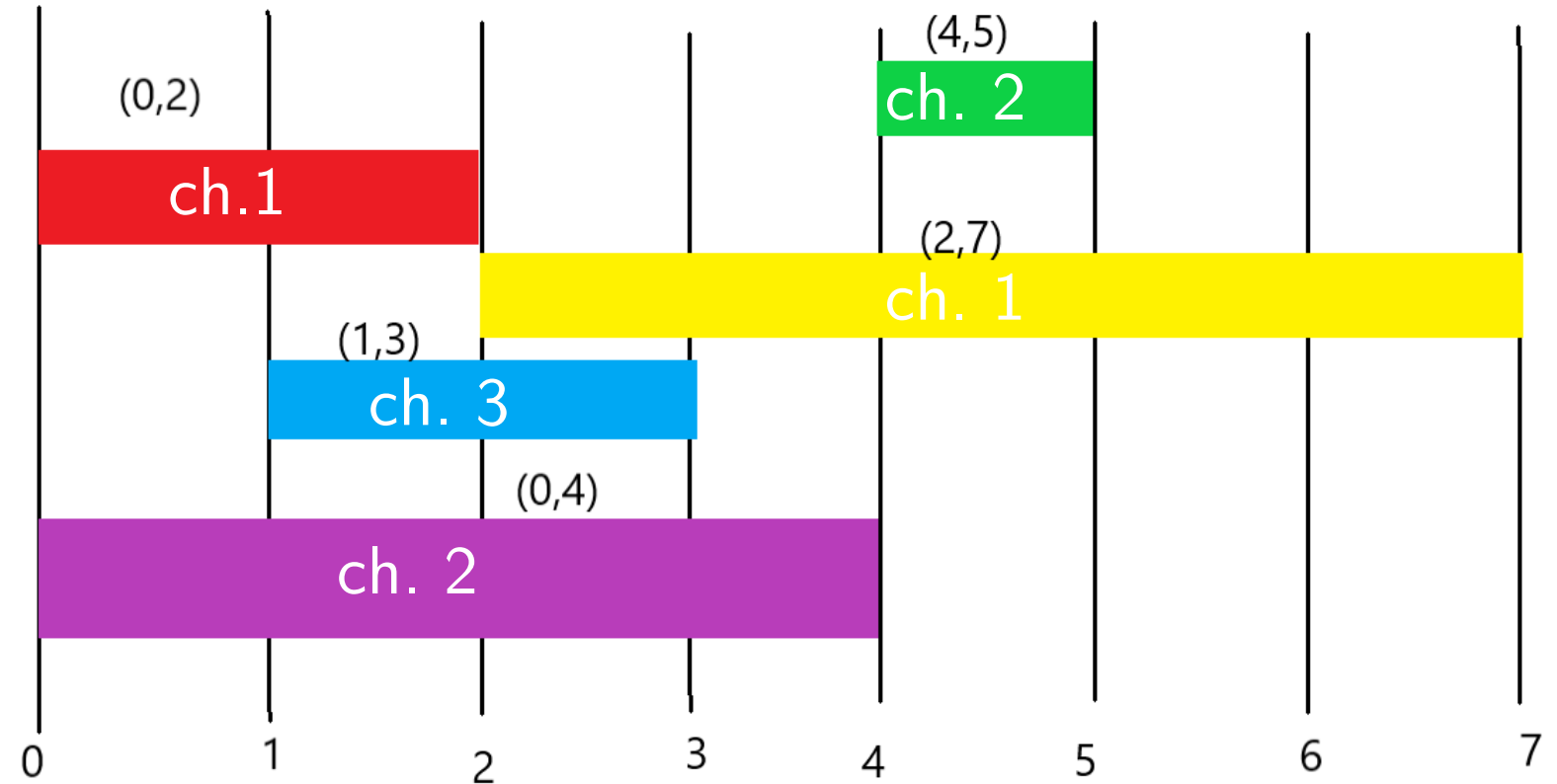
Equivalent problem: *tasks* instead of *events*, and *machines* instead of *channels*.

Scheduling, an example

Example of a set of events

$\{(4, 5), (0, 2), (2, 7), (1, 3), (0, 4)\}$

How many channels do we need?



Greedy Approach

Longest event first, then the next longest, etc. ?

For any greedy measure, the next event scheduled either has to

- Be scheduled in an active channel; requires no conflict with any event in that channel.
- Be scheduled in a **new** channel.

Greedy Rule: Take the longest first.

Idea: leave space for the shorter talks to “fill in” the unused spots

This sort of reasoning is common when coming up with greedy algorithms

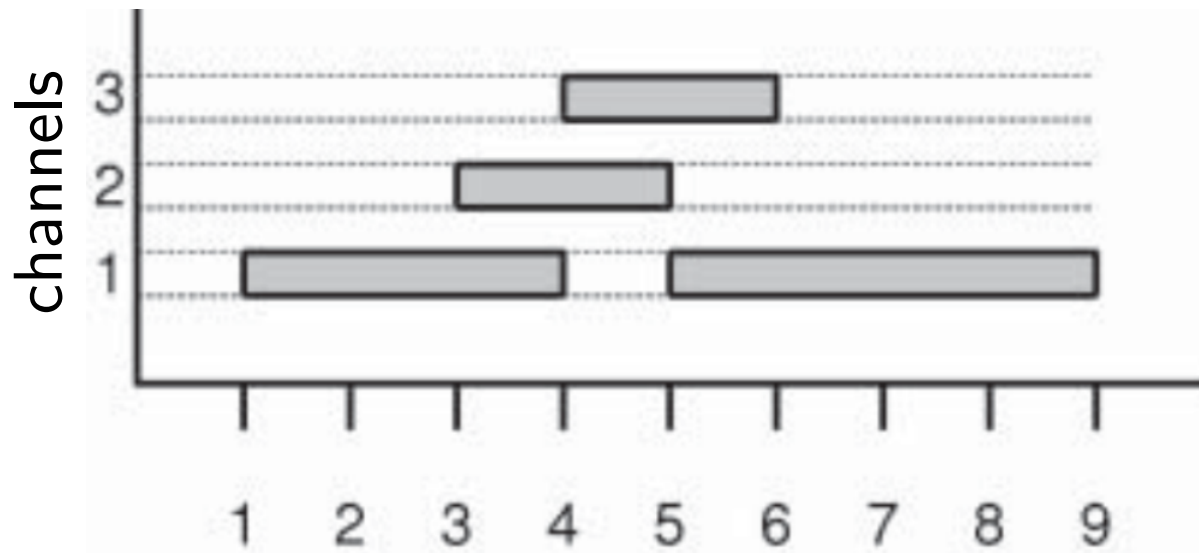
- Pick a likely-sounding rule
- See how it does

Approach #1: longest first

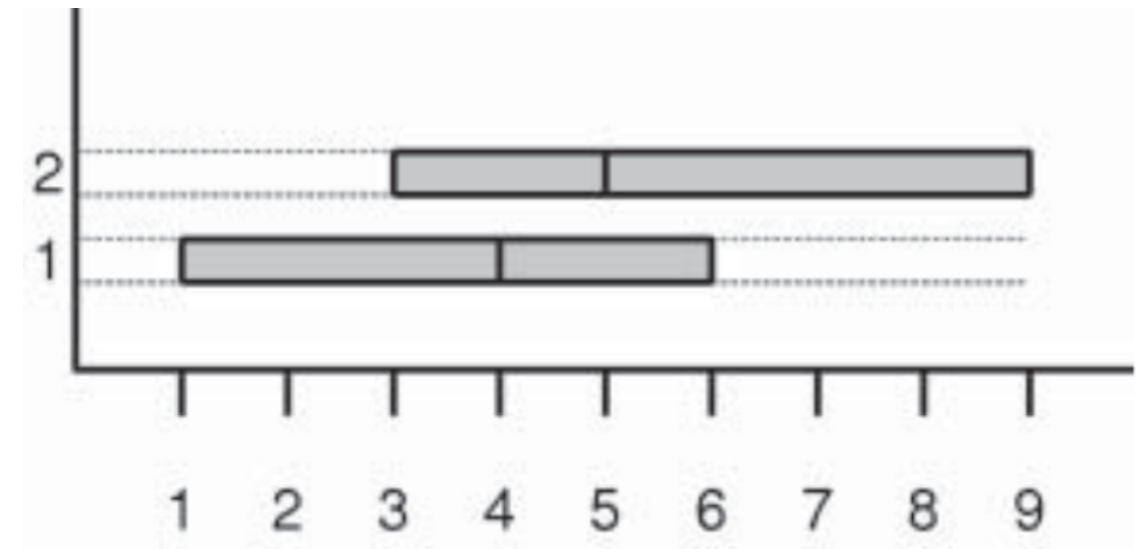
Input: $\{(5,9), (1,4), (3,5), (4,6)\}$ [sorted by length of time slot]

fit together :)

Longest first is not optimal :(



Scheduling Longest first (5,9) and (1,4) go in channel 1, but then (3,5) and (4,6) each need new channels.



Can do better!

Next Approach?

Other measures ... use the start or finish times to greedily fill the schedule?

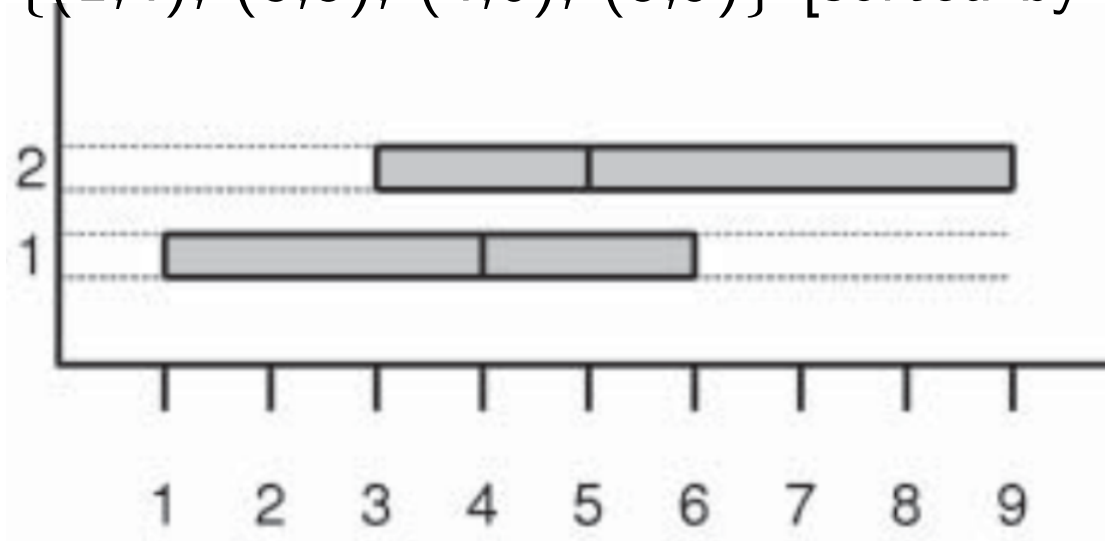
- Earliest to start first, then next earliest start time, etc.
- Earliest to finish first, then next earliest to finish, etc.
- Latest to finish first, then next latest, etc.
- ...

Greedy Rule: schedule events by earliest start time. (open a new channel when needed.)

Input: $\{(5,9), (1,4), (3,5), (4,6)\}$

$\rightsquigarrow \{(1,4), (3,5), (4,6), (5,9)\}$ [sorted by start time]

Looks promising!

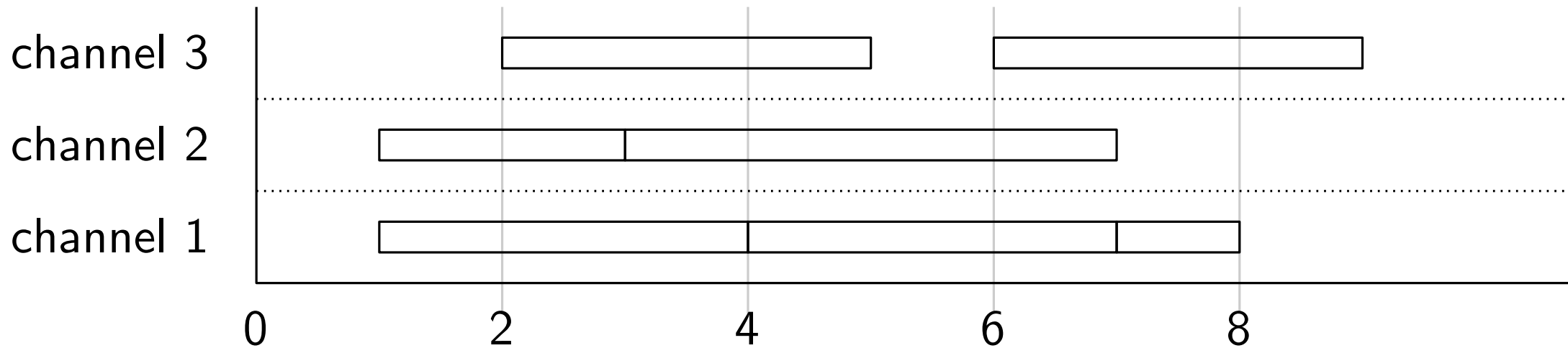


Exercise: write out the pseudocode for this algorithm and analyze the runtime.

Greedy Rule: schedule events by earliest start time.

Another example to try: (sorted by start time)

$(1,4)$, $(1,3)$, $(2,5)$, $(3,7)$, $(4,7)$, $(6,9)$, $(7,8)$



Let's prove this is optimal :)

Why does this work?

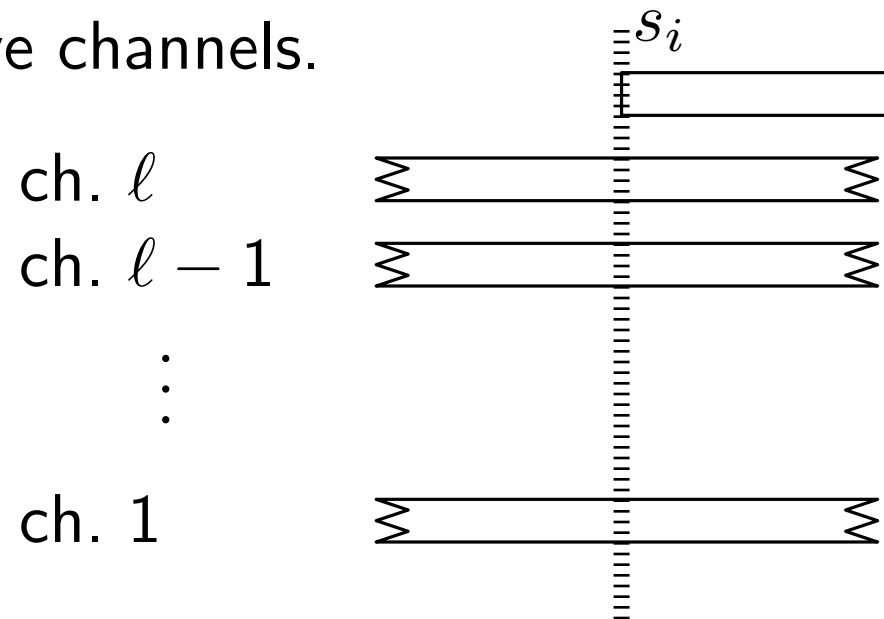
([GT 10.2] also provides a proof of this)

Let p be a schedule given by the **greedy rule**: schedule events by **earliest start time**.

Let p' be an optimal schedule, that is, using the fewest channels possible.

Suppose p uses k channels and p' uses ℓ channels. For a contradiction, assume $k > \ell$.

Let i be the first event planned in channel $\ell + 1$, i.e., $p(T_i) = \ell + 1$. Thus, T_i conflicts with all ℓ active channels.



This means we have $\ell + 1$ talks that pairwise conflict.

So, how can p' schedule use only ℓ channels?!?!?

This contradicts the assumption $k > \ell$.

Thus, the greedy rule is optimal!

Exercise What about those other orders by start and finish times, do they also work?

Problem: Task Scheduling [GT §10.2] Summary

- Streaming a collection of live **events** in the fewest **channels**.
- Given: a set of events by time slots $T_1, \dots, T_n$, where
 - each T_i has a start and finish time, (s_i, f_i) , $s_i < f_i$.
- Objective: Minimize the number of streaming channels needed to show all events.

Greedy Rule: schedule events into channels by earliest starting time.

(Vague) Implementation:

- Sort the events by start time. $s_1 \leq s_2 \leq \dots \leq s_n$.
- For i from 1 to n
 - If there is a channel where T_i “fits”, add T_i to this channel.
 - Else, create a new channel and add T_i to this channel.

How to do this efficiently?

This algorithm (optimally) solves the task scheduling problem.

Problem: Activity Selection [CLRS §16.1]

- Streaming a collection of live **events**, but now with only **one channel**.
- Given: a set of events by time slots $T_1, \dots, T_n$,
 - each T_i has a start and finish time, (s_i, f_i) , $s_i < f_i$.
- Objective: Schedule the **maximum** amount of events into **one channel**.
- Notes:
 - Do not optimize the time the channel needs to broadcast, only the number of events.
 - The output should indicate which events are scheduled.

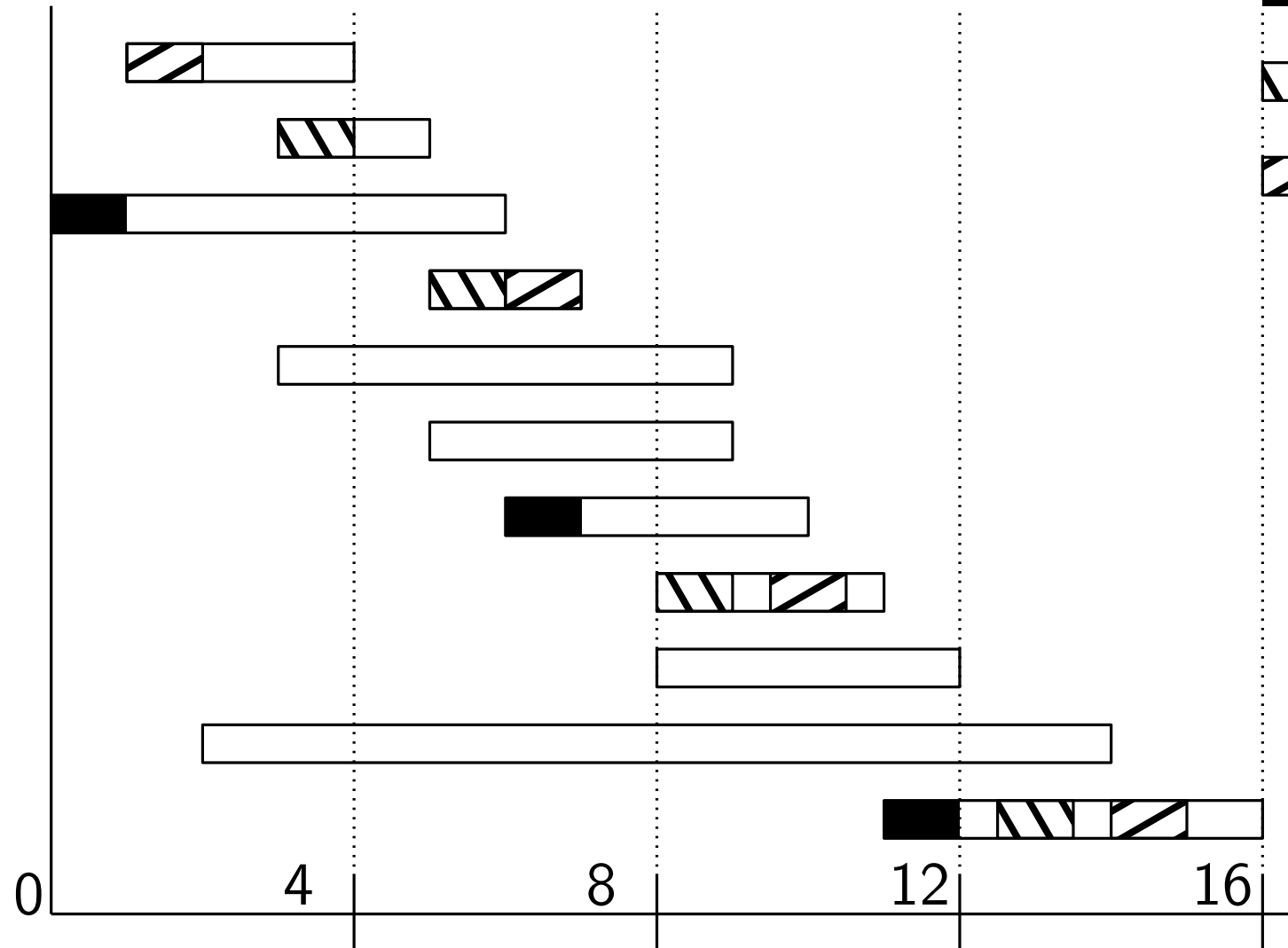
What is a schedule/plan? Function $p : \text{events} \rightarrow \{\text{yes}, \text{no}\}$ (schedule or not).

When is a schedule **feasible**? No **conflicting** events are set to “yes”.
 $(p(T_i) = p(T_j) = \text{yes}) \Rightarrow \text{either } f_i \leq s_j \text{ or } f_j \leq s_i.$

Equivalent problem: *activities* instead of *events*.

Greedy Rule Ideas

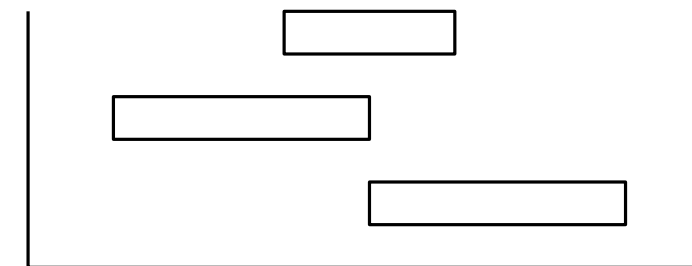
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16



Some ideas to try:

- ■ Earliest Start time 3
- ▨ ■ Longest first → Shortest first 4
- ▧ ■ Earliest Finish time 4

Break Shortest First?



(3, 5)

(1, 4), (4, 7)

Break Earliest Finish time?

Prove it works!

Earliest Finish Time is Optimal

Let p be a schedule given by the **greedy rule**: schedule events by **earliest finish time**.

Recall the input is a set of time slots $(s_1, f_1), \dots, (s_n, f_n)$ for the events.

Let the events be sorted in increasing order by finish time. That is:

$$f_1 \leq f_2 \leq \dots \leq f_n.$$

Note: An **optimal** schedule allocates the **maximum** amount of events in **one channel**.

Proof technique: **exchange argument**

Let p' be an optimal schedule that schedules the **most events in common** with p .

Case 1: p and p' are the same. The greedy rule is optimal :)

Case 2: p and p' are different.

Let i be the smallest index
where $p(T_i)$ and $p'(T_i)$ differ.

Case 2a: $p(T_i) = \text{yes}$, $p'(T_i) = \text{no}$.

Case 2b: $p(T_i) = \text{no}$, $p'(T_i) = \text{yes}$.

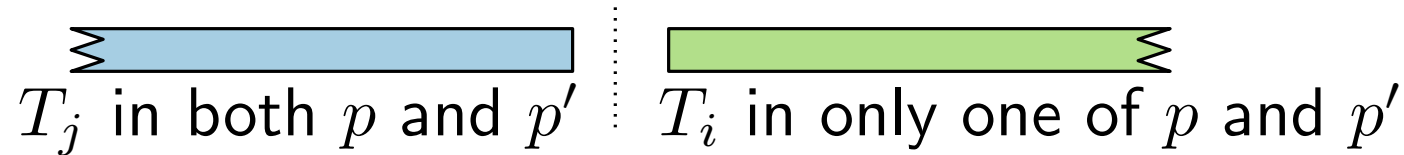
Earliest Finish Time is Optimal: Exchange Argument

Case 2: p [greedy] and p' [OPT] are different.

Thus, this rule is optimal :)

Let i be the smallest index where $p(T_i)$ and $p'(T_i)$ differ.

j be the largest index where $j < i$ and $p(T_j) = p'(T_j) = \text{yes}$. (most recent common pick)

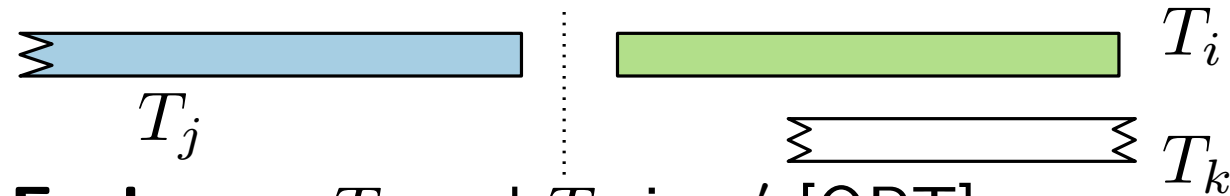


Case 2a: $p(T_i) = \text{yes}$, $p'(T_i) = \text{no}$.

Case 2b: $p(T_i) = \text{no}$, $p'(T_i) = \text{yes}$.

Let k be the smallest index where $k > j$ and $p'(T_k) = \text{yes}$. (next pick in OPT)

By choice of i , $k > i$. Thus, $f_i \leq f_k$.



Exchange T_i and T_k in p' [OPT].

Create: $p'' = p'$, then set $p''(T_i) = \text{yes}$, $p''(T_k) = \text{no}$. $\rightsquigarrow$ p'' optimal, but p'' has more in common with p than p'

Contradiction!

Let k be the smallest index where $k > j$ and $p(T_k) = \text{yes}$. (next greedy pick)

By choice of i , $k > i$. Thus, $f_i \leq f_k$.

Since $p'(T_i) = \text{yes}$, $s_i \geq f_j$.

Thus, $i < k$ and $s_i \geq f_j$.

$\Rightarrow$ [greedy] should have set $p(T_i) = \text{yes}$.

Contradiction!

Problem: Activity Selection [CLRS §16.1] Summary

- Streaming a collection of live **events**, but now with only **one channel**.
- Given: a set of events by time slots $T_1, \dots, T_n$,
 - each T_i has a start and finish time, (s_i, f_i) , $s_i < f_i$.
- Objective: Schedule the **maximum** amount of events into **one channel**.

Greedy Rule: schedule events into **the** channel by earliest finishing time.

(Vague) Implementation:

- Sort the events by finish time. $f_1 \leq f_2 \leq \dots \leq f_n$.
- For i from 1 to n
 - T_i “fits” into the channel, add T_i to the channel.
 - Else, skip T_i .

This algorithm (optimally) solves the activity scheduling problem.

General Remarks on Greedy

- **Natural** greedy algorithm:
 - sort input elements
 - for each element (in sorted order):
 - * keep it or throw it away, depending on problem constraints
- Proof of correctness structure: prove that each partial solution is **promising**, i.e., there is some optimal solution that **extends** it (that agrees with all greedy choices) – core of proof often involves an **exchange argument**.

Some more examples, but now with Graphs!

- Reminder of graphs and how we represent them.
- Event/Task/Activity Scheduling as graph problems.
- Minimum Spanning Trees [GT 15] [CLRS 23], Prim's algorithm [GT 15.3] [CLRS 23.2].
- Shortest Paths [GT 14] [CLRS 24], Dijkstra [GT 14.2] [CLRS 24.3]

Graphs: (road) networks, relation diagrams, etc.

What is a graph?

- graph G
- vertex set $V(G) = \{v_1, v_2, \dots, v_n\}$
- edge set $E(G) = \{e_1, e_2, \dots, e_m\}$,
where each edge is a pair of vertices.

Representation?

■ Set notation

$$V(G) = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9, v_{10}\}$$

$$E(G) = \{\{v_1, v_2\}, \{v_1, v_8\}, \{v_2, v_3\}, \{v_3, v_5\}, \{v_3, v_9\}, \\ \{v_3, v_{10}\}, \{v_4, v_5\}, \{v_4, v_6\}, \{v_4, v_9\}, \{v_5, v_8\}, \\ \{v_6, v_8\}, \{v_6, v_9\}, \{v_7, v_8\}, \{v_7, v_9\}, \{v_8, v_{10}\}, \\ \{v_9, v_{10}\}\}$$

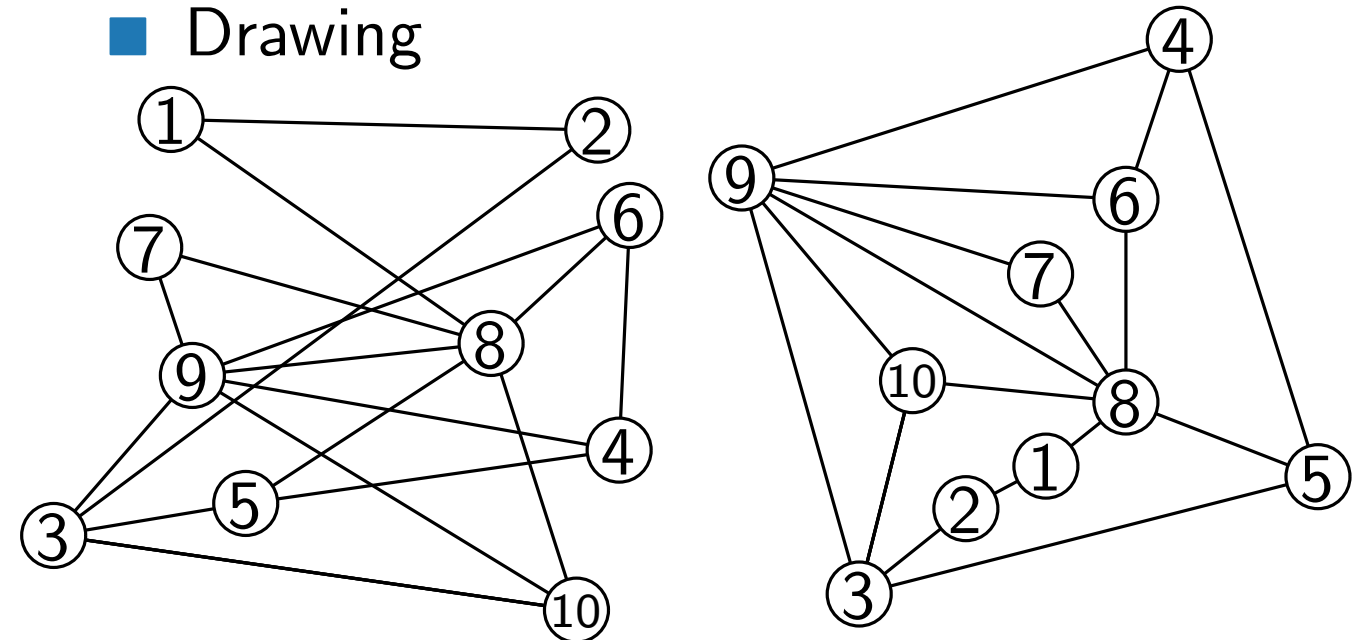
■ Adjacency list

$v_1:$	v_2, v_8	$v_6:$	v_4, v_8, v_9
$v_2:$	v_1, v_3	$v_7:$	v_8, v_9
$v_3:$	v_2, v_5, v_9, v_{10}	$v_8:$	$v_1, v_5, v_6, v_7, v_9, v_{10}$
$v_4:$	v_5, v_6, v_9	$v_9:$	$v_3, v_4, v_6, v_7, v_8, v_{10}$
$v_5:$	v_3, v_4, v_8	$v_{10}:$	v_3, v_8, v_9

■ Adjacency matrix

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

■ Drawing



Task Scheduling as a Graph Problem

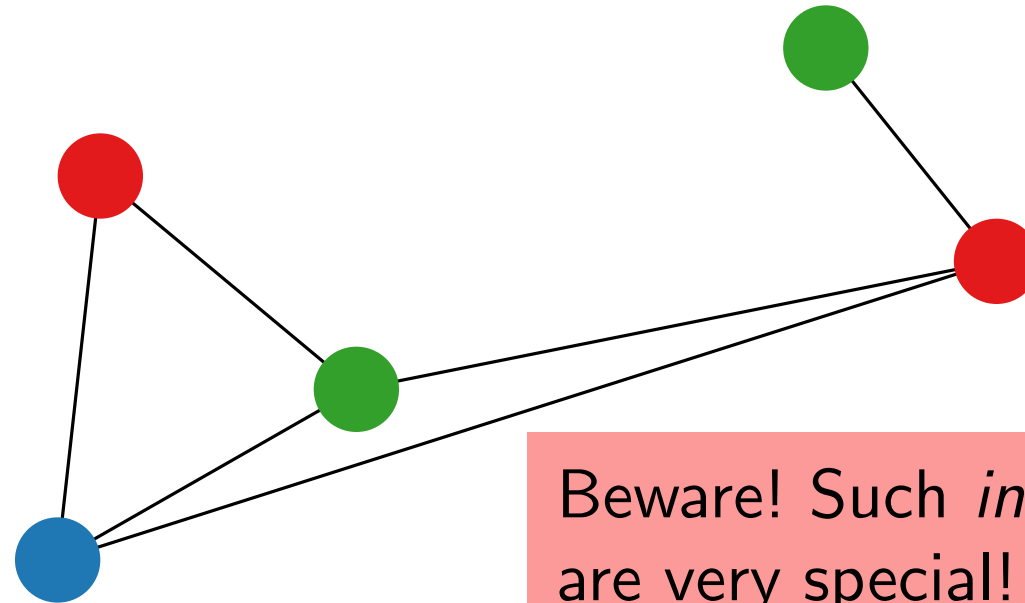
Example of a set of events/tasks
 $\{(4, 5), (0, 2), (2, 7), (1, 3), (0, 4)\}$

Create one vertex for each event.

Connect vertices u, v by an edge if they **conflict**, i.e., intervals overlap.

A valid schedule is now **graph coloring**:
color vertices so that no like-colored vertices are adjacent.

Using the fewest colorings now corresponds to using the fewest channels.



Beware! Such *interval* graphs are very special! In general, the fastest algorithms for graph coloring (2009) take $O(n \cdot 2^n)$ steps, for n -vertex graphs.

Activity Selection as a Graph Problem

Example of a set of events/activities

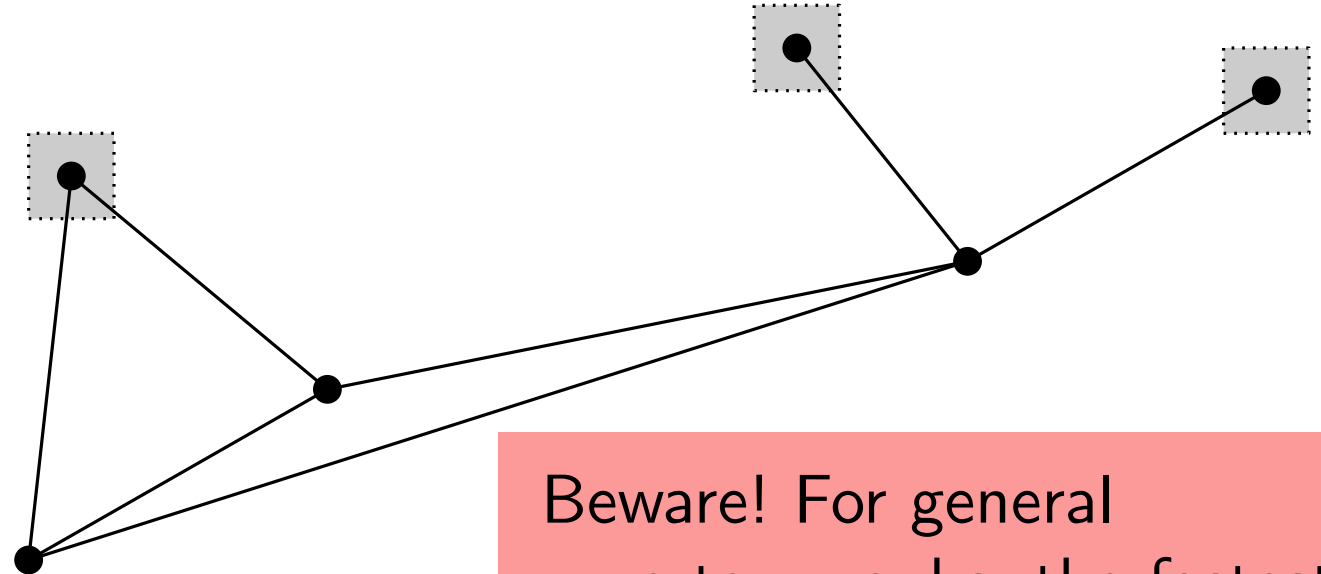
$\{(0, 2), (1, 3), (0, 4), (4, 5), (2, 7), (6, 7)\}$

Create one vertex for each event.

Connect vertices u, v by an edge if they **conflict**, i.e., intervals overlap.

A valid schedule is now an **independent set** of vertices: a set of vertices where no two form an edge.

Picking a maximum independent set of vertices corresponds to scheduling the most events.



Beware! For general n -vertex graphs, the fastest algorithms (2017) for maximum independent set take $O(1.1996^n)$ steps.

Problem: Efficient Sub-Network Construction

- Need to build a fibre optic network for my kingdom connecting all important locations.
- For ease of planning, cables are built along the road network.
 - The roads are given as segments connecting important locations, and
 - each segment has a cost to have a cable built there.
- I want to connect my kingdom at minimum total cost

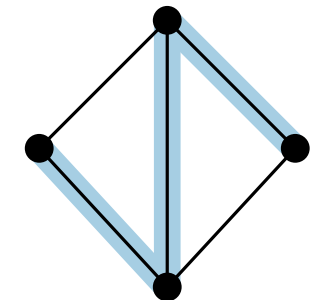
Represent road network as a **graph**: road segments are the edges, locations are the vertices.

Should our fibre network contain **cycles** ?

No, the solution should be a **tree**.

How do we ensure all locations have service?

The solution needs to **connect** all locations.



MINIMUM SPANNING TREE (MST)

Input: An n -vertex graph $G = (V, E)$, and positive edge costs $c : E \rightarrow \mathbb{R}_{>0}$.

Goal: Return a tree T that *spans* G , i.e., $V(T) = V$ and $E(T) \subset E$, where the total edge cost, $\sum_{e \in E(T)} c(e)$, is minimized.

Greedy Algorithm for MST (Prim), [GT §15, CLRS §23]

What are our partial solutions? Trees in our graph.

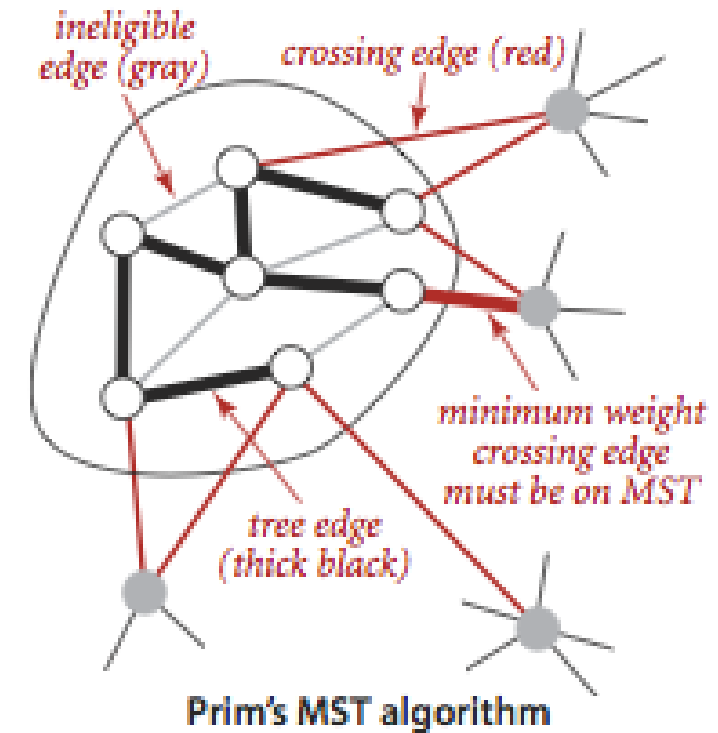
How do we “grow” a partial solution greedily?

Greedy Rule: Pick a **cheapest** (minimum cost) edge that **adds a new vertex** to the tree.

How can we do this efficiently?

Data structures to the rescue!

Maintain **eligible** edges to be added in a **heap**.

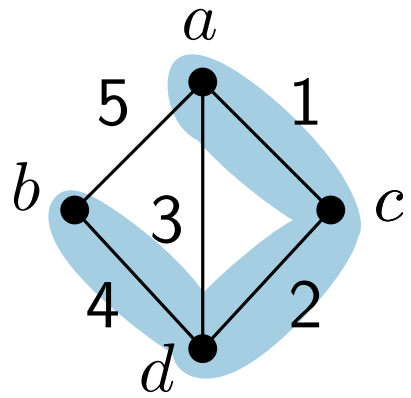


Exercises:

- Formalize this algorithm in pseudocode.
- Analyze the runtime. Think carefully about how you should represent the graph.

But wait, does that really work?

Quick sanity check:



iteration:	1	2	3	4
Vertices in our Tree	$\{a\}$	$\{a, c\}$	$\{a, c, d\}$	$\{a, c, d, b\}$
Edges (by cost) in the Heap:	$\{1, 3, 5\}$	$\{2, 3, 5\}$	$\{4, 5\}$	$\{\}$

Looks good for this example, but can we prove it in general?

Optimality of Prim's Greedy Rule

Greedy Rule: Pick a **cheapest** (minimum cost) edge that **adds a new vertex** to the tree.

Proof (that the greedy rule is optimal):

Let T be a spanning tree constructed by the greedy algorithm, and let T' be a minimum spanning tree.

Let $e = uv$ be the first edge added to T by the greedy algorithm that is not an edge of T' .

T' also connects the left blob with the right, say by e' .

But greedy took e instead of e' , so $c(e) \leq c(e')$.

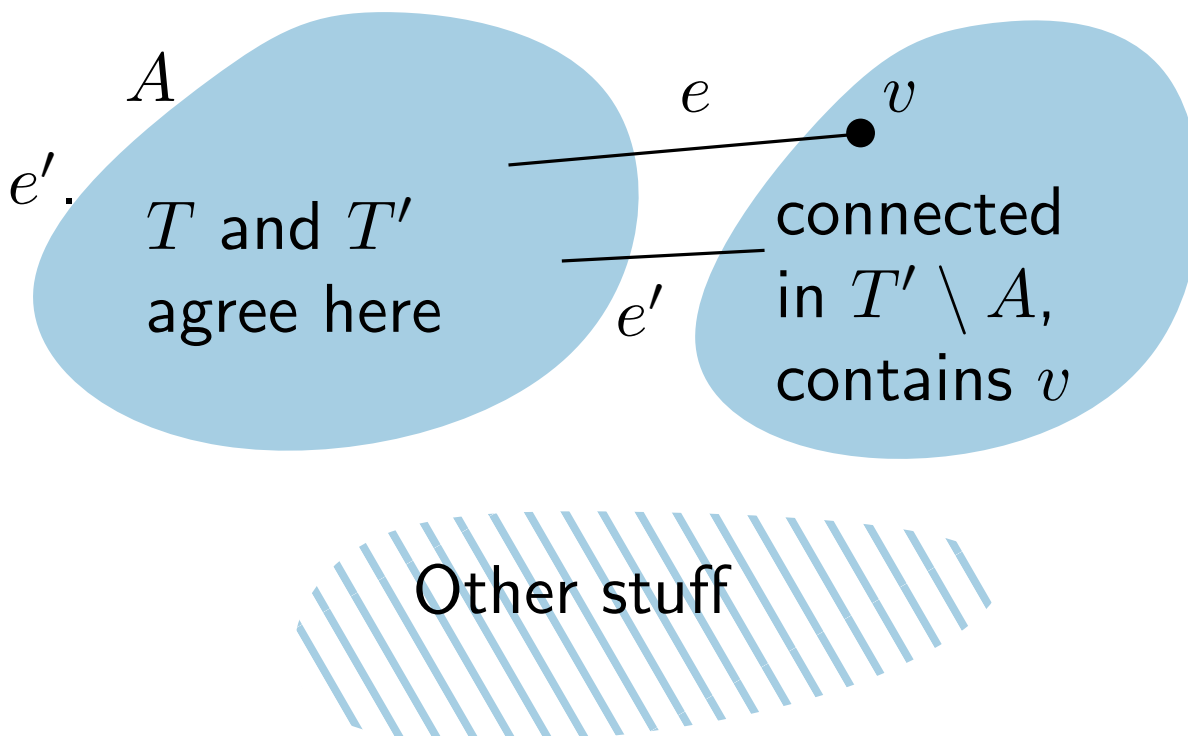
First suppose $c(e) < c(e')$

Let T'' be the result of swapping e' for e in T' .

T'' is a spanning tree.

But this contradicts T' having minimum cost!

How can we handle the case when $c(e) = c(e')$?



Correctness of Prim (part 2)

Proof:

Let T be a spanning tree constructed by the greedy algorithm, and let T' be a minimum spanning tree **such that T' and T share as many edges as possible**.

Let $e = uv$ be the first edge added to T by the greedy algorithm that is not an edge of T' .

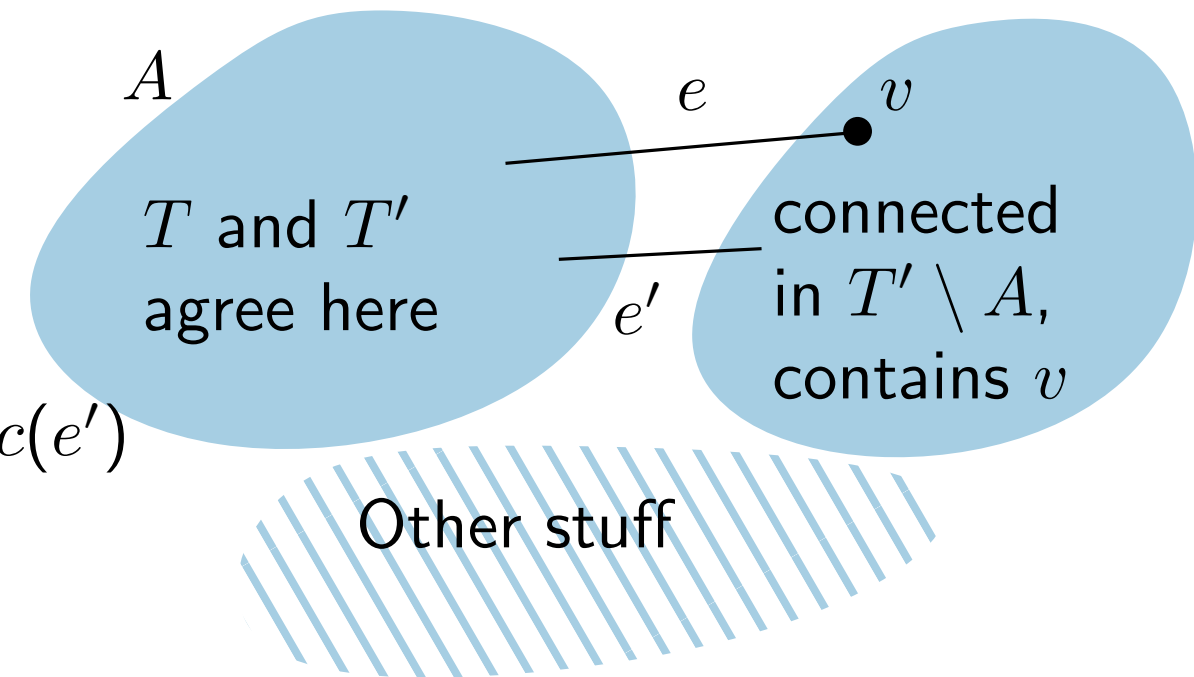
T' also connects the left blob to the right, say by e' .

Greedy took e instead of e' , so $c(e) < c(e')$, **or** $c(e) = c(e')$

Let T'' be the result of swapping e' for e in T' .

If $c(e') < c(e)$, contradiction as before.

If $c(e') = c(e)$, then T'' is also an MST, but now shares more edges with T than T' , contradicting the maximum commonality of T and T' . ■



Problem: Network Construction / MST, Summary

- Build a fibre optic network connecting all important locations along a road network (graph).
 - The roads are given as segments connecting important locations, and
 - each segment has a cost to have a cable built there.
- Connect locations at minimum total cost

MINIMUM SPANNING TREE (MST)

Input: An n -vertex graph $G = (V, E)$, and positive edge costs $c : E \rightarrow \mathbb{R}_{>0}$.

Goal: Return a tree T that *spans* G , i.e., $V(T) = V$ and $E(T) \subset E$, where the total edge cost, $\sum_{e \in E(T)} c(e)$, is minimized.

Greedy Rule: Pick a **cheapest** (minimum cost) edge that **adds a new vertex** to the tree.

Recall: **Exercise:** write out the pseudocode for this algorithm and analyze the runtime.

This algorithm (optimally) solves the MST problem.

Network Routing (Shortest Paths in Graphs) [CLRS §24]

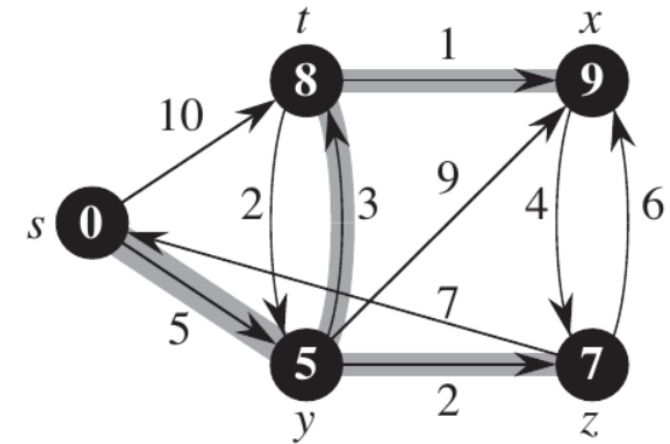
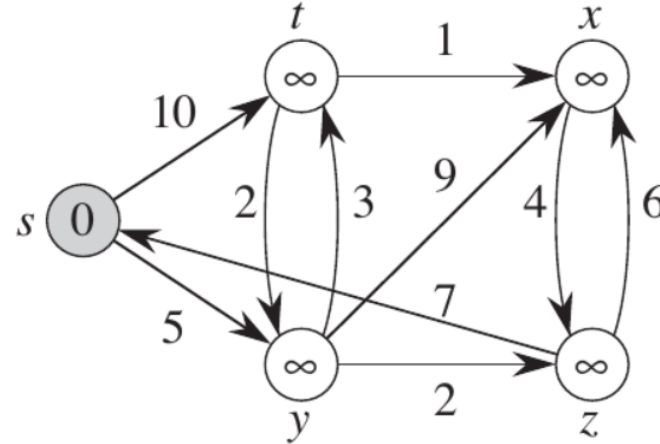
SINGLE-SOURCE SHORTEST PATHS

Input: An n -vertex **directed** graph $G = (V, E)$, and positive edge weights $w : E \rightarrow \mathbb{R}_{>0}$.
A starting vertex $s \in V$.

Goal: Calculate a shortest path from s to every other vertex v in G .

Idea: Prefixes of shortest paths need to be shortest paths.

$\leadsto$ shortest-path tree



Greedy Rule: Grow shortest-path tree by extending via to the **nearest** vertex to s .
(continue on supplemental slides)

Challenges in Greedy Design

- Different algorithms result from varying the objective function, and the starting conditions.
- How to decide which choices to make?
 - It is important that the start position and the rule for choosing a next move allow for *enough* configurations to be reached
- Deciding on an objective function:
 - Technically, the objective function only applies when the configuration is complete
 - One has to create a function to apply to incomplete configurations
 - Usually this is straightforward, but different choices can lead to different algorithms

General Remarks on Greedy

- **Basic** greedy algorithm:
 - sort input elements
 - for each element (in sorted order):
 - * keep it or throw it away, depending on problem constraints
- Proof of correctness structure: prove that each partial solution is **promising**, i.e., there is some optimal solution that **extends** it (that agrees with all greedy choices) – core of proof often involves an **exchange argument**.
- **General** greedy algorithm:
 - Until *finishing condition* [e.g., spanning tree is complete]
 - * Grow the partial solution: pick the element within the constraints that
 - minimizes additional cost [for minimization problems] or
 - maximizes additional profits [for maximization problems]

Exercises:

In these slides: (some with corresponding exercises in the textbooks)

1. Give a greedy algorithm for the **Picking Coins** problem that runs in $O(n \log t)$ time.
2. For the **Task Scheduling** problem
 - (a) Are the following greedy rules optimal? (if so prove it, otherwise give an example input to break it)
 - i. Earliest finish time first.
 - ii. Latest finish time first.
 - (b) For the greedy rule (earliest start time first) implement it in pseudocode and analyze the runtime. [GT 10.4, R-10.2]
3. For the **Minimum Spanning Tree** problem, implement the greedy rule (cheapest edge to grow the tree) in pseudocode and analyze the runtime. [CLRS 23.2-2]

Recommended from the textbooks:

[GT 10.4, A-10.1] ART GALLERY GUARDING. Given a line L representing a long, and real numbers $X = \{x_0, x_1, \dots, x_{n-1}\}$ specifying positions of paintings in this hallway. Suppose that a single guard can protect all the paintings within distance at most 1 of his or her position (on both sides). Design an algorithm for finding a placement of guards that uses the minimum number of guards to guard all the paintings with positions in X . (see [CLRS 16.2-5] for same problem phrased differently)

[CLRS 16.1-3] Not just any greedy approach to the activity-selection problem produces a maximum-size set of mutually compatible activities. Give an example to show that the approach of selecting the activity of least duration from among those that are compatible with previously selected activities does not work. Do the same for the approaches of always selecting the compatible activity that overlaps the fewest other remaining activities and always selecting the compatible remaining activity with the earliest start time.

[CLRS 23-4] Alternative minimum-spanning-tree algorithms In this problem, we give pseudocode for three different algorithms. Each one takes a connected graph and a weight function as input and returns a set of edges T . For each algorithm, either prove that T is a minimum spanning tree or prove that T is not a minimum spanning tree. Also describe the most efficient implementation of each algorithm, whether or not it computes a minimum spanning tree.

a. MAYBE-MST-A(G, w)

```

1  sort the edges into nonincreasing order of edge weights  $w$ 
2   $T = E$ 
3  for each edge  $e$ , taken in nonincreasing order by weight
4      if  $T - \{e\}$  is a connected graph
5           $T = T - \{e\}$ 
6  return  $T$ 
```

b. MAYBE-MST-B(G, w)

```

1   $T = \emptyset$ 
2  for each edge  $e$ , taken in arbitrary order
3      if  $T \cup \{e\}$  has no cycles
4           $T = T \cup \{e\}$ 
5  return  $T$ 
```

c. MAYBE-MST-C(G, w)

```

1   $T = \emptyset$ 
2  for each edge  $e$ , taken in arbitrary order
3       $T = T \cup \{e\}$ 
4      if  $T$  has a cycle  $c$ 
5          let  $e'$  be a maximum-weight edge on  $c$ 
6           $T = T - \{e'\}$ 
7  return  $T$ 
```

Next Week

More algorithmic techniques: Divide and Conquer

e.g., as in Merge Sort.

How to analyze the runtime of recursive methods (e.g., as arising from Divide and Conquer)

The MASTER theorem